

Program 2: Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins

Step 1: To create a Maven project run the following command in terminal:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

This command generates a basic Maven project structure with a sample App.java and AppTest.java.

```
samiksha@samiksha:~$ mkdir report
samiksha@samiksha:~$ cd report
samiksha@samiksha:~/report$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 MB at 5.8 MB/s)
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] -----
[INFO] Parameter: basedir, Value: /home/samiksha/report
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/samiksha/report/MyMavenApp
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 13.271 s
[INFO] Finished at: 2026-03-17T18:56:11+05:30
[INFO]
[INFO] -----
samiksha@samiksha:~/report$
```

This “BUILD SUCCESS” will confirm that the project is created successfully

Step 2: Go to the project directory using command **cd MyMavenApp**

Step 3: Open and edit pom.xml file using command **gedit pom.xml**

- Go to <https://mvnrepository.com>
- Search for a dependency “junit”
- Select the latest version and copy the dependency XML snippet.
- Paste it under the <dependencies> section in pom.xml
- Updated pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>MyMavenApp</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>
<name>MyMavenApp</name>
```

```

<url>http://maven.apache.org</url>
<!-- Java version configuration -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
<!-- Dependencies -->
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
</dependencies>
<!-- Build Plugins -->
<build>
<plugins>
<!-- Compiler Plugin -->
<plugin>
<groupId>org.apache.maven.plugins</groupId> <artifactId>maven-compiler-
plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>11</source>
<target>11</target>
</configuration>
</plugin>
<!-- Surefire Plugin for Testing -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<!-- JAR Plugin with Entry Point -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<version>3.1.0</version>
<configuration>
<archive>
<manifestEntries>
<Main-Class>com.example.App</Main-Class>
</manifestEntries>
</archive>
</configuration>

```

```
</plugin>
</plugins>
</build>
</project>
```

Step 4: Build the project using command **mvn clean install**

```
samiksha@samiksha:~/report/MyMavenApp$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenApp >-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenApp ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/samiksha/report/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenApp ---
[INFO] Building jar: /home/samiksha/report/MyMavenApp/target/MyMavenApp-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenApp ---
[INFO] Installing /home/samiksha/report/MyMavenApp/target/MyMavenApp-1.0-SNAPSHOT.jar to /home/samiksha/.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.jar
[INFO] Installing /home/samiksha/report/MyMavenApp/pom.xml to /home/samiksha/.m2/repository/com/example/MyMavenApp/1.0-SNAPSHOT/MyMavenApp-1.0-SNAPSHOT.pom
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO]
[INFO] Total time: 9.566 s
[INFO] Finished at: 2026-03-17T19:08:24+05:30
[INFO] -----
```

The “Build Success” message ensures that our Maven project is built successfully.

Step 5: To run the Maven application type the command:

java -jar target/MyMavenApp-1.0-SNAPSHOT.jar

- After completing the execution, the output will look like:

```
samiksha@samiksha:~/report/MyMavenApp$ java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
Hello World!
samiksha@samiksha:~/report/MyMavenApp$
```

This will show output as the code present in App.java file we can edit the text to get any other output.

Creating a Maven Project with Guava and Commons-IO

Google Guava Dependency

Guava is a set of core libraries developed by Google to fill gaps in Java's standard libraries. It is used for:

- Collections enhancements: Multimap, BiMap, Table, ImmutableList, etc.
- Functional programming utilities: Predicates, Functions, Suppliers.
- Caching: Built-in caching mechanisms (CacheBuilder) like a mini in-memory cache.
- String utilities: Splitter, Joiner, CharMatcher.
- Concurrency: ListenableFuture, Service, and other helper classes.
- Hashing and hashing strategies: Useful for consistent hashing (e.g., Bloom filters).

Overall, Guava simplifies and improves many core Java patterns and utilities.

Apache Commons IO Dependency

Apache Commons IO is focused purely on input/output utilities, especially for file and stream handling. It is used for:

- File operations: Easily read/write files with FileUtils.
- Stream utilities: IOUtils.copy(), IOUtils.toString() — simplifies stream handling.
- Directory monitoring: FileAlterationObserver for detecting changes in a folder.
- File filtering: WildcardFileFilter, SuffixFileFilter.

Overall, Commons IO reduces boilerplate and errors when working with files or streams.

Maven Project with Guava and Commons-IO Dependencies

Create a Maven Project Using Terminal by following command:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
samiksha@samiksha:~/report$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/samiksha/report
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenGuavaApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/samiksha/report/MyMavenGuavaApp
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 5.360 s
[INFO] Finished at: 2026-03-17T19:29:09+05:30
[INFO]
[INFO] -----
samiksha@samiksha:~/report$
```

Now Navigate to the Project Directory using **cd MyMavenGuavaApp**

```
samiksha@samiksha:~/report$ cd MyMavenGuavaApp
samiksha@samiksha:~/report/MyMavenGuavaApp$
```

Edit the pom.xml File and add the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>MyMavenGuavaApp</artifactId>
<packaging>jar</packaging>
<version>1.0-SNAPSHOT</version>
<name>MyMavenGuavaApp</name>
<url>http://maven.apache.org</url>
<dependencies>
<dependency>
```

```

    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
</dependency>

<!-- https://mvnrepository.com/artifact/com.google.guava/guava -->
<dependency>
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>33.4.0-jre</version>
</dependency>

<!-- https://mvnrepository.com/artifact/commons-io/commons-io -->
<dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.18.0</version>
</dependency>

</dependencies>

<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>

<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>

<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-jar-plugin</artifactId>
    <version>3.1.0</version>
    <configuration>

```

```

        <archive>
            <manifestEntries>
                <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
        </archive>
    </configuration>
</plugin>

</plugins>
</build>
</project>

```

Modify App.java

Got to App.java using **cd src/main/java/com/example/** and **gedit App.java** to type the file with following code:

```
package com.example;
```

```
/**
```

```
 * Hello world!
```

```
 *
```

```
 */
```

```
// Import FileUtils class from Apache Commons IO library
```

```
// This is used to perform file operations like copying files
```

```
import org.apache.commons.io.FileUtils;
```

```
// Import File class to represent file and directory pathnames
```

```
import java.io.File;
```

```
// Import IOException to handle input-output errors
```

```
import java.io.IOException;
```

```
// Import ImmutableList from Google Guava library
```

```
// ImmutableList creates a list that cannot be modified after creation
```

```
import com.google.common.collect.ImmutableList;
```

```
// Define the main class
```

```
public class App
```

```
{
```

```
    // Main method - program execution starts from here
```

```
    public static void main( String[] args )
```

```

{
    // Create an immutable list of strings (fruits)
    // This list cannot be changed (no add/remove allowed)
    ImmutableList<String> fruits = ImmutableList.of("Apple", "Banana", "Cherry");

    // Print the list of fruits to the console
    System.out.println(fruits);

    // Create a File object representing the source file
    File sourceFile = new File("source.txt");

    // Create a File object representing the destination file
    File destFile = new File("destination.txt");

    try {
        // Copy the contents of source.txt to destination.txt
        // Using Apache Commons IO FileUtils class
        FileUtils.copyFile(sourceFile, destFile);

        // Print success message if file is copied
        System.out.println("File copied successfully!");
    }
    catch (IOException e) {
        // If any error occurs during file copying,
        // this block will handle the exception
        // Print error message
        System.err.println("Error occurred while copying file: " + e.getMessage());
    }
}

```

Build the Project using the command **mvn clean install** command

```

[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenGuavaApp ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.2.0/maven-archiver-3.2.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.2.0/maven-archiver-3.2.0.pom (4.3 kB at 144 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.pom (4.9 kB at 189 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/3.5/plexus-archiver-3.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/3.5/plexus-archiver-3.5.pom (4.8 kB at 185 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-5.0/plexus-5.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-5.0/plexus-5.0.pom (21 kB at 790 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.0.0/plexus-io-3.0.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.0.0/plexus-io-3.0.0.pom (4.5 kB at 175 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.14/commons-compress-1.14.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.14/commons-compress-1.14.pom (13 kB at 505 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/42/commons-parent-42.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/42/commons-parent-42.pom (68 kB at 2.2 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/tukaani/xz/1.6/xz-1.6.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/tukaani/xz/1.6/xz-1.6.pom (1.9 kB at 73 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.1.0/plexus-utils-3.1.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.1.0/plexus-utils-3.1.0.pom (4.7 kB at 155 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.2.0/maven-archiver-3.2.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.2.0/maven-archiver-3.2.0.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/3.5/plexus-archiver-3.5.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/3.5/plexus-archiver-3.5.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.0.0/plexus-io-3.0.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.0.0/plexus-io-3.0.0.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.14/commons-compress-1.14.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.14/commons-compress-1.14.jar (530 kB at 5.4 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.2.0/maven-shared-utils-3.2.0.jar (165 kB at 1.6 MB/s)
[INFO] Building jar: /home/unified/Devops/MyMavenGuavaApp/target/MyMavenGuavaApp-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenGuavaApp ---
[INFO] Installing /home/unified/Devops/MyMavenGuavaApp/target/MyMavenGuavaApp-1.0-SNAPSHOT.jar to /home/unified/.m2/repository/com/example/MyMavenGuavaApp/1.0-SNAPSHOT/MyMavenGuavaApp-1.0-SNAPSHOT.jar
[INFO] Installing /home/unified/Devops/MyMavenGuavaApp/pom.xml to /home/unified/.m2/repository/com/example/MyMavenGuavaApp/1.0-SNAPSHOT/MyMavenGuavaApp-1.0-SNAPSHOT.pom
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.093 s
[INFO] Finished at: 2025-05-28T21:31:52+05:30
[INFO]

```

This compiles the project and creates a JAR in the target/ folder.

Run the Application:

If java-jar command give an error which is most probable, use the following command
mvn exec:java -Dexec.mainClass="com.example.App"

```

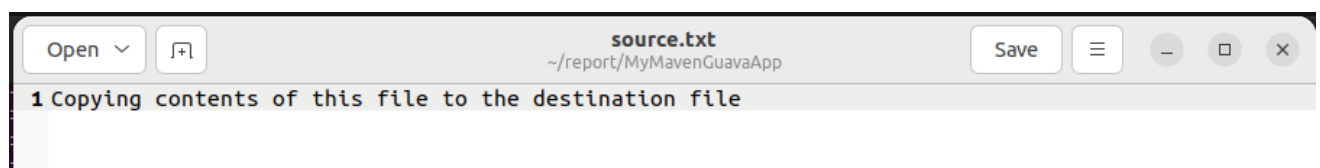
samiksha@samiksha:~/report/MyMavenGuavaApp$ mvn exec:java -Dexec.mainClass="com.example.App"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenGuavaApp >-----
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec-maven-plugin:3.6.3:java (default-cli) @ MyMavenGuavaApp ---
[Apple, Banana, Cherry]
Error occurred while copying file: Source 'source.txt' does not exist
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.559 s
[INFO] Finished at: 2026-03-17T20:28:33+05:30
[INFO]

```

This command gives error as

Error while copying file:Source ‘source.txt’ does not exist

Now, Create a Sample File to copy the content to destination file



Now Run the command **mvn exec:java -Dexec.mainClass="com.example.App"** again

```
samiksha@samiksha:~/report/MyMavenGuavaApp$ gedit source.txt
samiksha@samiksha:~/report/MyMavenGuavaApp$ mvn exec:java -Dexec.mainClass="com.example.App"
[INFO] Scanning for projects...
[INFO] -----< com.example:MyMavenGuavaApp >-----
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO] --- exec-maven-plugin:3.6.3:java (default-cli) @ MyMavenGuavaApp ---
[Apple, Banana, Cherry]
File copied successfully!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.326 s
[INFO] Finished at: 2026-03-17T20:31:05+05:30
[INFO] -----
samiksha@samiksha:~/report/MyMavenGuavaApp$
```

The output shows file copied successfully.

We can see newly created **destination.txt** using gedit destination.txt if you are still in Project's directory

```
samiksha@samiksha:~/report/MyMavenGuavaApp$ ls
destination.txt pom.xml source.txt src target
samiksha@samiksha:~/report/MyMavenGuavaApp$
```

File content is same as the content in source file.

Program 3: Working with Selenium: Maven Project To Connect To Web Servers

Using Selenium Dependency

1.Sauce.demo webpage

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false**

```
samiksha@samiksha:~/report$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] -----
[INFO] Parameter: basedir, Value: /home/samiksha/report
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenSeleniumApp01
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/samiksha/report/MyMavenSeleniumApp01
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.456 s
[INFO] Finished at: 2026-03-17T20:40:07+05:30
[INFO] -----
```

Step 2: Before we connect to web servers we need to have chrome driver installed. To download chrome driver

To check the chrome version type the command **google-chrome --version** in the terminal

```
samiksha@samiksha:~/report$ google-chrome --version
Google Chrome 145.0.7632.116
samiksha@samiksha:~/report$
```

extract the version and store it in a variable **CHROME_VERSION**

```
samiksha@samiksha:~/report$ CHROME_VERSION=$(google-chrome --version | awk '{print $3}' | cut -d'.' -f1-3)
samiksha@samiksha:~/report$
```

extracts and transforms JSON data

LATEST_DRIVER=\$(curl -s "https://googlechromelabs.github.io/chrome-for-testing/latest-patch-versions-per-build.json" | jq -r ".builds[\"\$CHROME_VERSION\"].version")

do install curl using the command

sudo apt install curl

wget https://storage.googleapis.com/chrome-for-testing-public/\$LATEST_DRIVER/linux64/chromedriver-linux64.zip

Extract and move it to /usr/local/bin/:

unzip chromedriver-linux64.zip

sudo mv chromedriver-linux64/chromedriver /usr/local/bin/chromedriver

sudo chmod +x /usr/local/bin/chromedriver

Check if ChromeDriver is installed correctly:

```
samiksha@samiksha:~/report$ chromedriver --version
ChromeDriver 145.0.7632.117 (de4e5d6e13b995f5e04137970c0e8ec456017467-refs/branch-heads/7632@{#3101})
samiksha@samiksha:~/report$
```

Step 3 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenSeleniumApp01</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenSeleniumApp01</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
    <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.29.0</version>
    </dependency>
  </dependencies>
  <!-- Build: Configuring plugins and build settings -->
  <build>
    <plugins>
      <!-- Example: Maven Compiler Plugin to compile Java code -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>
        <configuration>
          <source>1.7</source>
          <target>1.7</target>
        </configuration>
      </plugin>
      <!-- Example: Maven Surefire Plugin to run tests -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>2.22.2</version>
```

```

</plugin>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>

    <artifactId>maven-jar-plugin</artifactId>
    <version>3.0.1</version>
    <configuration>
        <archive>
            <manifestEntries>
                <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
        </archive>
    </configuration>

</plugin>

```

```

</plugins>

```

```

</build>

```

```

</project>

```

Step 4: Edit the App.java

```

package com.example;

```

```

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

```

```

/**

```

```

 * Hello world!

```

```

 *

```

```

 */

```

```

public class App

```

```

{

```

```

    public static void main( String[] args )

```

```

    {

```

```

        WebDriver driver=new ChromeDriver();

```

```

        driver.get("https://www.saucedemo.com/");

```

```

        driver.manage().window().maximize();

```

```

        driver.findElement(By.id("user-name")).sendKeys("standard_user");

```

```

        driver.findElement(By.id("password")).sendKeys("secret_sauce");

```

```

        driver.findElement(By.id("login-button")).click();

```

```

    }

```

```

}

```

Step 5: build and run the maven application

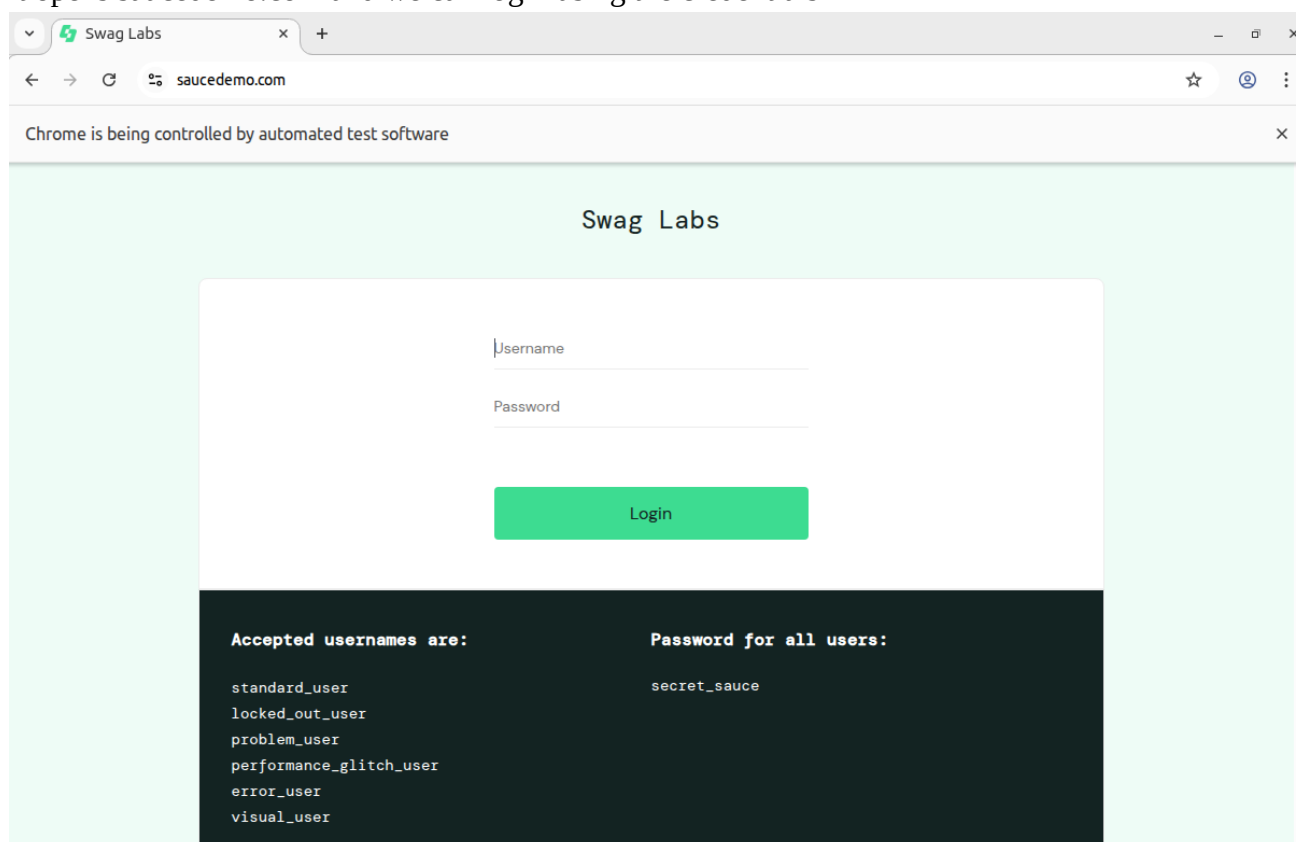
Type **mvn clean install** in the terminal

```

samiksha@samiksha:~/report/MyMavenSeleniumApp01$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenSeleniumApp01 >-----
[INFO] Building MyMavenSeleniumApp01 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenSeleniumApp01 ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenSeleniumApp01 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/samiksha/report/MyMavenSeleniumApp01/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenSeleniumApp01 ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/samiksha/report/MyMavenSeleniumApp01/target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 11.900 s
[INFO] Finished at: 2026-03-17T21:18:01+05:30
[INFO]

```

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**
it opens saucedemo.com and we can login using the credentials



2. practiceautomation.com

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenPracticeAutomation -**

DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
samiksha@samiksha:~/Devops60$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenPracticeAutomation -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.456 s
[INFO] Finished at: 2026-03-17T20:40:07+05:30
[INFO] -----
```

Step 2 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenPracticeAutomationApp</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenPracticeAutomationApp</name>
  <url>http://maven.apache.org</url>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.29.0</version>
    </dependency>
  </dependencies>

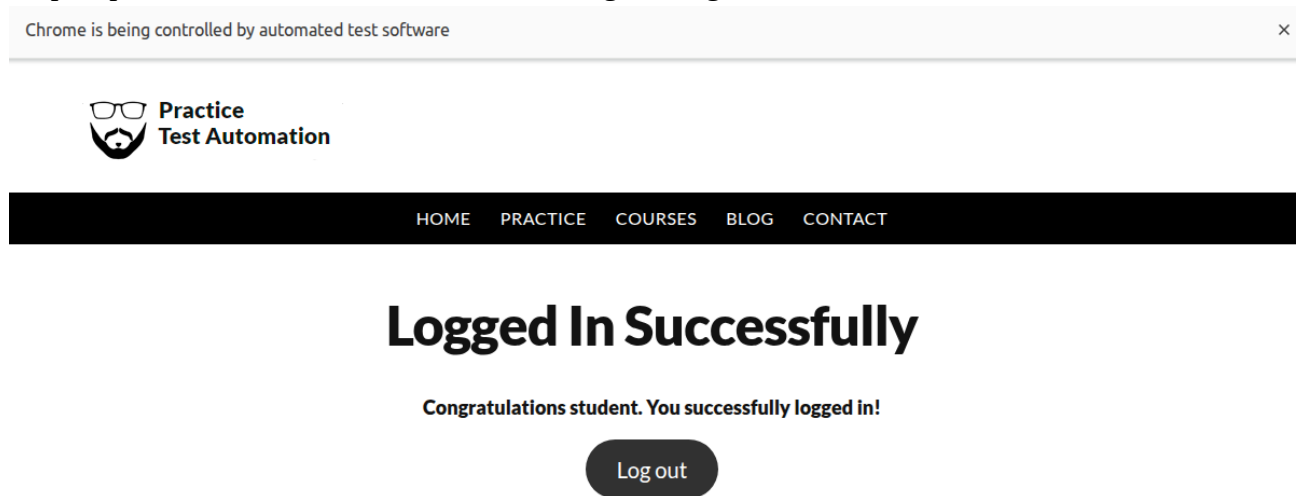
  <!-- Build: Configuring plugins and build settings -->
  <build>
    <plugins>
      <!-- Example: Maven Compiler Plugin to compile Java code -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>
        <configuration>
          <source>1.7</source>
          <target>1.7</target>
        </configuration>
      </plugin>
      <!-- Example: Maven Surefire Plugin to run tests -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>2.22.2</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.0.1</version>
        <configuration>
          <archive>
            <manifestEntries>
              <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Step 3: Edit the App.java

```
package com.example;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class App
{
    public static void main( String[] args )
    {
        WebDriver driver = new ChromeDriver();
        driver.get("https://practicetestautomation.com/practice-test-login/");
        driver.manage().window().maximize();
        driver.findElement(By.id("username")).sendKeys("student");
        driver.findElement(By.id("password")).sendKeys("Password123");
        driver.findElement(By.id("submit")).click();
    }
}
```

Step 4: build and run the maven application

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**
it opens practicetestautomation.com and we can login using the credentials



3.automationexercise.com

Step 1: create a maven project on terminal by typing the command **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenAutomationExercise -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false**

```
samksha@samksha:~/Devops60/MyMavenPracticeAutomationApp$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenExerciseAutomationApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
INFO] -----  
INFO] BUILD SUCCESS  
INFO] -----  
INFO] Total time: 4.456 s  
INFO] Finished at: 2026-03-17T20:40:07+05:30  
INFO] -----
```

Step 2 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-  
v4_0_0.xsd">
```

```
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.example</groupId>  
  <artifactId>MyMavenExerciseAutomationApp</artifactId>  
  <packaging>jar</packaging>  
  <version>1.0-SNAPSHOT</version>  
  <name>MyMavenExerciseAutomationApp</name>  
  <url>http://maven.apache.org</url>
```

```
  <dependencies>
```

```
    <dependency>
```

```
      <groupId>junit</groupId>
```

```
      <artifactId>junit</artifactId>
```

```
      <version>3.8.1</version>
```

```
      <scope>test</scope>
```

```
    </dependency>
```

```
    <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
```

```
    <dependency>
```

```
      <groupId>org.seleniumhq.selenium</groupId>
```

```
      <artifactId>selenium-java</artifactId>
```

```
      <version>4.29.0</version>
```

```
    </dependency>
```

```
    <dependency>
```

```
      <groupId>io.github.bonigarcia</groupId>
```

```
      <artifactId>webdrivermanager</artifactId>
```

```
      <version>5.8.0</version>
```

```
    </dependency>
```

```
  </dependencies>
```

```
  <!-- Build: Configuring plugins and build settings -->
```

```
  <build>
```

```
    <plugins>
```

```
    <!-- Example: Maven Compiler Plugin to compile Java code -->
```

```
    <plugin>
```

```
      <groupId>org.apache.maven.plugins</groupId>
```

```
      <artifactId>maven-compiler-plugin</artifactId>
```



```

<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>

    <artifactId>maven-jar-plugin</artifactId>
    <version>3.0.1</version>
    <configuration>
        <archive>
            <manifestEntries>
                <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
        </archive>
    </configuration>
</plugin>
</plugins>
</build>
</project>

```

Step 3: Edit the App.java

```
package com.example;
```

```

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;

```

```

import io.github.bonigarcia.wdm.WebDriverManager;
import java.time.Duration;

```

```

public class App {
    public static void main(String[] args) {

```

```

WebDriverManager.chromedriver().setup();
WebDriver driver = new ChromeDriver();
driver.manage().window().maximize();

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

try {
    driver.get("https://automationexercise.com");

    // Click Products page

    wait.until(ExpectedConditions.elementToBeClickable(By.xpath("//a[@href='/products']"))).click();

    // Find first Add to Cart button
    WebElement addToCart = wait.until(ExpectedConditions.elementToBeClickable(
        By.xpath("//a[@data-product-id='1']")));

    // Scroll and click using JavaScript (avoids ad overlay)
    ((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);",
addToCart);
    ((JavascriptExecutor) driver).executeScript("arguments[0].click();", addToCart);

    // Wait for confirmation modal
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("cartModal")));

    // Click View Cart
    wait.until(ExpectedConditions.elementToBeClickable(By.linkText("View
Cart"))).click();

    System.out.println("Product successfully added to cart!");

} catch (Exception e) {
    System.out.println("TEST FAILED: " + e.getMessage());
}

System.out.println("Browser will remain open for inspection.");
}
}

```

Step 4: build and run the maven application

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**
it opens automationexercise.com and automatically adds a product to cart



Proceed To Checkout

Item	Description	Price	Quantity	Total
	Blue Top Women > Tops	Rs. 500	1	Rs. 500

4. All 3 exercises executed using firefox

Step 1: create a maven project on terminal by typing the command `mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenFireFox -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false`

```
samiksha@samiksha:~/Devops60$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenFireFox -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.456 s
[INFO] Finished at: 2026-03-17T20:40:07+05:30
[INFO] -----
```

Step 2 : edit the pom.xml file and add the dependencies

The pom.xml has the code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId>
```

```
<artifactId>MyMavenExerciseAutomationApp</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
```

```
<packaging>jar</packaging>
```

```
<name>MyMavenExerciseAutomationApp</name>
```

```
<!-- Java Version -->
```

```
<properties>
```

```
  <maven.compiler.source>17</maven.compiler.source>
```

```
  <maven.compiler.target>17</maven.compiler.target>
```

```
</properties>
```

```

<dependencies>

<!-- Selenium -->
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.29.0</version>
</dependency>

<!-- WebDriverManager (handles GeckoDriver automatically) -->
<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>5.8.0</version>
</dependency>

<!-- JUnit (optional, for testing) -->
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>

</dependencies>

<build>
  <plugins>

    <!-- Compiler Plugin -->
    <plugin>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
    </plugin>

    <!-- Exec Plugin (to run your main class) -->
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.1.0</version>
      <configuration>
        <mainClass>com.example.App</mainClass>
      </configuration>
    </plugin>

  </plugins>

```

</build>

</project>

Step 3: Edit the App.java

```
package com.example;
```

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
```

```
public class App {
```

```
    public static void main(String[] args) {
```

```
        WebDriver driver = new FirefoxDriver();
```

```
        // ----- 1. SauceDemo Login -----
        driver.get("https://www.saucedemo.com/");
        driver.manage().window().maximize();
```

```
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();
```

```
        // ----- 2. Practice Test Login -----
        driver.get("https://practicetestautomation.com/practice-test-login/");
        driver.manage().window().maximize();
```

```
        driver.findElement(By.id("username")).sendKeys("student");
        driver.findElement(By.id("password")).sendKeys("Password123");
        driver.findElement(By.id("submit")).click();
```

```
        // ----- 3. Automation Exercise Search -----
        driver.get("https://automationexercise.com/products");
        driver.manage().window().maximize();
```

```
        driver.findElement(By.id("search_product")).sendKeys("Blue Top");
        driver.findElement(By.id("submit_search")).click();
```

```
        // driver.quit(); // optional
```

```
    }
```

Step 4: build and run the maven application

To execute type the command **mvn exec:java -Dexec.mainClass="com.example.App"**

This executes all the three exercises together.

